

Lab Assignment

C17 — STL `vector<int>` — Core Operations

`push_back` • `pop_back` • `emplace_back` • `front` • `back` • `insert` • `erase` • `clear` • `size` • `resize` • `swap` • `assign`

1. Objective

This lab provides hands-on practice with the most commonly used methods of the STL `vector<int>` container. Each experiment isolates a specific group of methods so you can observe their behavior precisely. Together the experiments cover the full lifecycle of a vector: creation, population, access, modification, resizing, and destruction of contents.

By the end of this lab you will be able to:

- ▶ Define a vector using an initializer list, `push_back()`, and `assign()`.
- ▶ Access elements using `operator[]`, `at()`, `front()`, and `back()`.
- ▶ Traverse using array-style index notation, forward iterators, and range-for loops.
- ▶ Add elements at the end with `push_back()` and `emplace_back()`.
- ▶ Remove the last element with `pop_back()`.
- ▶ Insert and remove elements at any position with `insert()` and `erase()`.
- ▶ Remove all elements with `clear()`.
- ▶ Query and adjust the size with `size()` and `resize()`.
- ▶ Exchange two vectors with `swap()` and replace all contents with `assign()`.

2. Quick Reference – Vector Methods

Method	Description
<code>v.push_back(x)</code>	Appends <code>x</code> to the end. Vector grows by 1.
<code>v.pop_back()</code>	Removes the last element. Vector shrinks by 1. Returns nothing.
<code>v.emplace_back(x)</code>	Constructs <code>x</code> in place at the end. Avoids a copy for class objects.
<code>v.front()</code>	Returns a reference to the FIRST element. Undefined if vector is empty.
<code>v.back()</code>	Returns a reference to the LAST element. Undefined if vector is empty.
<code>v[i]</code>	Access element at index <code>i</code> . No bounds checking — undefined if out of range.
<code>v.at(i)</code>	Access element at index <code>i</code> . Throws <code>out_of_range</code> if <code>i</code> is invalid.
<code>v.insert(pos, x)</code>	Inserts <code>x</code> BEFORE the iterator position <code>pos</code> .
<code>v.erase(pos)</code>	Removes the element at iterator position <code>pos</code> . Returns iterator to next element.
<code>v.erase(first,last)</code>	Removes all elements in the half-open range <code>[first, last)</code> .
<code>v.clear()</code>	Removes all elements. <code>size()</code> becomes 0; capacity is unchanged.
<code>v.size()</code>	Returns the current number of elements (type <code>size_t</code>).
<code>v.empty()</code>	Returns true if <code>size() == 0</code> .
<code>v.resize(n)</code>	Grows or shrinks to <code>n</code> elements. New slots are initialized to 0.
<code>v.resize(n, val)</code>	Grows or shrinks to <code>n</code> elements. New slots are filled with <code>val</code> .
<code>v.swap(other)</code>	Exchanges the full contents of <code>v</code> and <code>other</code> in $O(1)$ time.
<code>v.assign(n, val)</code>	Replaces ALL contents with <code>n</code> copies of <code>val</code> .
<code>v.assign({...})</code>	Replaces ALL contents with the values in the initializer list.
<code>v.begin()</code>	Iterator to the first element.
<code>v.end()</code>	Iterator to one past the last element (sentinel — do not dereference).
<code>v.rbegin()</code>	Reverse iterator to the last element.
<code>v.rend()</code>	Reverse iterator to one before the first element (sentinel).

3. Required Experiments

Experiment 01 — Define and Populate

Three ways to create and populate a `vector<int>`. Each method has different use cases — study when each is appropriate.

Tasks:

- ▶ Create `v1` using an initializer list = { 10, 20, 30, 40, 50 }.
- ▶ Create `v2` empty, then add elements one at a time using `push_back()`.
- ▶ Create `v3` using `assign(5, 99)` to fill it with five copies of 99.
- ▶ Print each vector and its `size()` after creation.

```
void experiment01() {
    cout << "=== Experiment 01: Define and Populate ===\n";

    // Method 1 -- initializer list
    vector<int> v1 = { 10, 20, 30, 40, 50 };

    // Method 2 -- push_back()
    vector<int> v2;
    for (int i = 1; i <= 5; i++)
        v2.push_back(i * 11);           // adds 11, 22, 33, 44, 55

    // Method 3 -- assign(count, value)
    vector<int> v3;
    v3.assign(5, 99);                   // five copies of 99

    cout << "v1 size=" << v1.size() << " [ ";
    for (int x : v1) cout << x << " "; cout << "]\n";

    cout << "v2 size=" << v2.size() << " [ ";
    for (int x : v2) cout << x << " "; cout << "]\n";

    cout << "v3 size=" << v3.size() << " [ ";
    for (int x : v3) cout << x << " "; cout << "]\n";
}
```

Experiment 02 — Traverse: Array Notation and Iterators

Show all four traversal styles on the same vector. Iterator style is preferred by STL algorithms; range-for is the most concise for reading.

Tasks:

- ▶ Traverse using `v[i]` with an index variable (array-style).
- ▶ Traverse using a forward iterator from `v.begin()` to `v.end()`.
- ▶ Traverse backward using a reverse iterator from `v.rbegin()` to `v.rend()`.
- ▶ Traverse using a range-for loop (simplest form).

```
void experiment02() {
    cout << "=== Experiment 02: Traversal ===\n";

    vector<int> v = { 10, 20, 30, 40, 50 };

    // Style 1 -- array-style index
    cout << "Array      : ";
    for (int i = 0; i < (int)v.size(); i++)
        cout << "v[" << i << "]= " << v[i] << " ";
    cout << endl;

    // Style 2 -- forward iterator
    cout << "Forward    : ";
    for (auto it = v.begin(); it != v.end(); ++it)
        cout << *it << " ";
    cout << endl;

    // Style 3 -- reverse iterator
    cout << "Backward   : ";
    for (auto it = v.rbegin(); it != v.rend(); ++it)
        cout << *it << " ";
    cout << endl;

    // Style 4 -- range-for (most concise)
    cout << "Range-for: ";
    for (int x : v)
        cout << x << " ";
    cout << endl;
}
```

Experiment 03 — front(), back(), push_back(), pop_back()

`front()` and `back()` give direct references to the first and last elements without iterators. `push_back()` grows the vector by one at the end; `pop_back()` shrinks it by one — it returns nothing so save the value first if you need it.

Tasks:

- ▶ Start with `v = { 10, 20, 30, 40, 50 }`.
- ▶ Print `v.front()` and `v.back()` to confirm first and last elements.
- ▶ Modify the first element via `v.front() = 99` and print the result.
- ▶ Append three values using `push_back()` — print size after each.
- ▶ Remove elements one at a time using `pop_back()` — save and print the value before each removal.
- ▶ Call `pop_back()` until the vector is empty and print `size()` to confirm.

```
void experiment03() {
    cout << "=== Experiment 03: front, back, push_back, pop_back ===\n";

    vector<int> v = { 10, 20, 30, 40, 50 };
    cout << "Initial    : [ "; for (int x : v) cout << x << " "; cout << "]\n";
    cout << "front()=" << v.front() << "    back()=" << v.back() << "\n";

    // Modify via front()
    v.front() = 99;
    cout << "After front()=99: [ ";
    for (int x : v) cout << x << " "; cout << "]\n";

    // push_back() -- grow at the end
    v.push_back(60);
    v.push_back(70);
    v.push_back(80);
    cout << "After 3x push_back: size=" << v.size() << "    [ ";
    for (int x : v) cout << x << " "; cout << "]\n";

    // pop_back() -- shrink from the end
    // Save the value BEFORE removing it -- pop_back returns void!
    while (!v.empty()) {
        cout << "    Removing back=" << v.back()
             << "    size before=" << v.size() << "\n";
        v.pop_back();
    }
    cout << "After all pop_backs: size=" << v.size() << "\n";
}
```

Experiment 04 — `emplace_back()` vs `push_back()`

`emplace_back()` constructs the element directly inside the vector memory — no temporary object is created. For `int` the difference is invisible; for class objects it avoids an extra copy or move. This experiment demonstrates both methods side by side.

Tasks:

- ▶ Start with an empty vector.
- ▶ Add five values alternating between `push_back()` and `emplace_back()`.
- ▶ Confirm the results are identical regardless of which method was used.
- ▶ Add the same values to a second vector using only `emplace_back()` and compare.

```
void experiment04() {
    cout << "=== Experiment 04: emplace_back vs push_back ===\n";

    vector<int> v1, v2;

    // Alternating push_back and emplace_back
    v1.push_back(10);
    v1.emplace_back(20);
    v1.push_back(30);
    v1.emplace_back(40);
    v1.push_back(50);

    // All emplace_back
    v2.emplace_back(10);
    v2.emplace_back(20);
    v2.emplace_back(30);
    v2.emplace_back(40);
    v2.emplace_back(50);

    cout << "v1 (mixed)      : [ ";
    for (int x : v1) cout << x << " "; cout << "]\n";

    cout << "v2 (emplace)     : [ ";
    for (int x : v2) cout << x << " "; cout << "]\n";

    cout << "Results equal : " << (v1 == v2 ? "YES" : "NO") << "\n";
}
```

Experiment 05 — insert() at Any Position

`insert(pos, val)` places a new element before the element pointed to by `pos`. Every element at and after the insertion point is shifted right, so all existing iterators at or after `pos` are invalidated after the call.

Tasks:

- ▶ Start with `v = { 10, 20, 40, 50 }` (30 is intentionally missing).
- ▶ Insert 30 between 20 and 40 using `v.begin() + 2` as the position.
- ▶ Insert 5 at the beginning using `v.begin()`.
- ▶ Insert 99 at the end using `v.end()` (equivalent to `push_back`).
- ▶ Print the vector and `size()` after each insertion.

```
void experiment05() {
    cout << "=== Experiment 05: insert() ===\n";

    vector<int> v = { 10, 20, 40, 50 };
    cout << "Initial          : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // Insert 30 at index 2 (before element 40)
    v.insert(v.begin() + 2, 30);
    cout << "After insert(30) : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // Insert 5 at the front
    v.insert(v.begin(), 5);
    cout << "After insert(5)   : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // Insert 99 at the end
    v.insert(v.end(), 99);
    cout << "After insert(99)  : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    cout << "Final size       : " << v.size() << "\n";
}
```

Experiment 06 — erase() and clear()

`erase()` removes one element or a range and returns an iterator to the next surviving element. This return value must be captured when erasing inside a loop. `clear()` removes all elements but does not release reserved capacity.

Tasks:

- ▶ Start with `v = { 5, 10, 20, 30, 20, 40, 50 }`.
- ▶ Erase the first element using `erase(v.begin())`.
- ▶ Erase the last two elements using `erase(v.end()-2, v.end())`.
- ▶ Erase all occurrences of 20 using an iterator loop that captures the return value of `erase()`.
- ▶ Call `clear()` and verify `size()` is 0 and `empty()` returns true.

```
void experiment06() {
    cout << "=== Experiment 06: erase() and clear() ===\n";

    vector<int> v = { 5, 10, 20, 30, 20, 40, 50 };
    cout << "Initial          : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // Erase the first element
    v.erase(v.begin());
    cout << "After erase(front)   : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // Erase the last two elements
    v.erase(v.end() - 2, v.end());
    cout << "After erase(last 2)   : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // Erase all occurrences of 20 -- must capture erase() return value!
    for (auto it = v.begin(); it != v.end(); ) {
        if (*it == 20)
            it = v.erase(it); // erase returns iterator to next element
        else
            ++it;
    }
    cout << "After erase all 20s   : [ "; for(int x:v) cout<<x<<" "; cout<<"]\n";

    // clear() -- remove everything
    v.clear();
    cout << "After clear(): size=" << v.size()
        << "   empty=" << (v.empty() ? "true" : "false") << "\n";
}
```


Experiment 07 — size() and resize()

`size()` returns the current element count. `resize(n)` grows or shrinks the vector to exactly `n` elements. When growing without a fill value new slots are zero-initialized. When shrinking, elements beyond position `n` are permanently discarded.

Tasks:

- ▶ Start with `v = { 10, 20, 30 }`. Print `size()`.
- ▶ Grow with `resize(6)` — new elements are 0. Print result.
- ▶ Grow with `resize(9, 99)` — new elements are 99. Print result.
- ▶ Shrink with `resize(4)` — elements beyond index 3 are gone. Print result.
- ▶ Call `resize(0)` and compare with `clear()` — both produce size 0.

```
void experiment07() {
    cout << "=== Experiment 07: size() and resize() ===\n";

    vector<int> v = { 10, 20, 30 };
    cout << "Initial      size=" << v.size() << " [ ";
    for (int x : v) cout << x << " "; cout << "]\n";

    // Grow to 6 -- new elements default to 0
    v.resize(6);
    cout << "resize(6)    size=" << v.size() << " [ ";
    for (int x : v) cout << x << " "; cout << "]\n";

    // Grow to 9 -- new elements filled with 99
    v.resize(9, 99);
    cout << "resize(9,99) size=" << v.size() << " [ ";
    for (int x : v) cout << x << " "; cout << "]\n";

    // Shrink to 4 -- elements at indices 4..8 are discarded
    v.resize(4);
    cout << "resize(4)    size=" << v.size() << " [ ";
    for (int x : v) cout << x << " "; cout << "]\n";

    // resize(0) vs clear()
    v.resize(0);
    cout << "resize(0)    size=" << v.size()
        << " (same effect as clear)\n";
}
```

Experiment 08 — swap() and assign()

`swap()` exchanges the complete contents of two vectors in $O(1)$ time — no elements are moved or copied.
`assign()` replaces every element in the vector with a new set of values, changing the size if necessary.

Tasks:

- ▶ Create `v1 = { 1, 2, 3 }` and `v2 = { 10, 20, 30, 40, 50 }`.
- ▶ Print both before the swap — note the different sizes.
- ▶ Call `v1.swap(v2)` and print both — confirm sizes also swapped.
- ▶ Call `v1.assign(3, 77)` — confirm all elements replaced with 77.
- ▶ Call `v2.assign({ 100, 200, 300 })` using an initializer list.
- ▶ Print both vectors after assign.

```
void experiment08() {
    cout << "=== Experiment 08: swap() and assign() ===\n";

    vector<int> v1 = { 1, 2, 3 };
    vector<int> v2 = { 10, 20, 30, 40, 50 };

    cout << "Before swap:\n";
    cout << "  v1 size=" << v1.size() << "  [ ";
    for (int x : v1) cout << x << " "; cout << "]\n";
    cout << "  v2 size=" << v2.size() << "  [ ";
    for (int x : v2) cout << x << " "; cout << "]\n";

    // swap() -- O(1), no element copies
    v1.swap(v2);

    cout << "After swap:\n";
    cout << "  v1 size=" << v1.size() << "  [ ";
    for (int x : v1) cout << x << " "; cout << "]\n";
    cout << "  v2 size=" << v2.size() << "  [ ";
    for (int x : v2) cout << x << " "; cout << "]\n";

    // assign(n, val) -- replace all with n copies of val
    v1.assign(3, 77);
    cout << "v1 after assign(3,77): [ ";
    for (int x : v1) cout << x << " "; cout << "]\n";

    // assign with initializer list
    v2.assign({ 100, 200, 300 });
    cout << "v2 after assign list: [ ";
    for (int x : v2) cout << x << " "; cout << "]\n";
}
```

main() — Call All Experiments

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    experiment01();
    experiment02();
    experiment03();
    experiment04();
    experiment05();
    experiment06();
    experiment07();
    experiment08();
    cout << "\nAll done!\n";
    return 0;
}
```

4. Expected Output (partial)

```
=== Experiment 01: Define and Populate ===
v1 size=5  [ 10 20 30 40 50 ]
v2 size=5  [ 11 22 33 44 55 ]
v3 size=5  [ 99 99 99 99 99 ]

=== Experiment 02: Traversal ===
Array      : v[0]=10 v[1]=20 v[2]=30 v[3]=40 v[4]=50
Forward    : 10 20 30 40 50
Backward   : 50 40 30 20 10
Range-for: 10 20 30 40 50

=== Experiment 03: front, back, push_back, pop_back ===
Initial    : [ 10 20 30 40 50 ]
front()=10  back()=50
After front()=99: [ 99 20 30 40 50 ]
After 3x push_back: size=8  [ 99 20 30 40 50 60 70 80 ]
Removing back=80  size before=8
Removing back=70  size before=7
... (continues until empty)
After all pop_backs: size=0

=== Experiment 04: emplace_back vs push_back ===
v1 (mixed)   : [ 10 20 30 40 50 ]
v2 (emplace) : [ 10 20 30 40 50 ]
Results equal: YES

=== Experiment 05: insert() ===
Initial      : [ 10 20 40 50 ]
After insert(30) : [ 10 20 30 40 50 ]
After insert(5)  : [ 5 10 20 30 40 50 ]
After insert(99) : [ 5 10 20 30 40 50 99 ]
Final size      : 7

=== Experiment 06: erase() and clear() ===
Initial      : [ 5 10 20 30 20 40 50 ]
After erase(front) : [ 10 20 30 20 40 50 ]
After erase(last 2) : [ 10 20 30 20 ]
After erase all 20s : [ 10 30 ]
After clear(): size=0  empty=true

=== Experiment 07: size() and resize() ===
Initial      size=3  [ 10 20 30 ]
resize(6)     size=6  [ 10 20 30 0 0 0 ]
resize(9,99)  size=9  [ 10 20 30 0 0 0 99 99 99 ]
resize(4)     size=4  [ 10 20 30 0 ]
resize(0)     size=0  (same effect as clear)

=== Experiment 08: swap() and assign() ===
Before swap:
  v1 size=3  [ 1 2 3 ]
  v2 size=5  [ 10 20 30 40 50 ]
After swap:
  v1 size=5  [ 10 20 30 40 50 ]
  v2 size=3  [ 1 2 3 ]
v1 after assign(3,77): [ 77 77 77 ]
v2 after assign list:  [ 100 200 300 ]
```

5. Common Mistakes

Pitfalls to avoid

- ▶ **Calling `front()` or `back()` on an empty vector** — both are undefined behavior. Always check `empty()` first.
- ▶ **Calling `pop_back()` and ignoring the lost value** — `pop_back()` returns `void`. Save `v.back()` before calling it if you need the value.
- ▶ **Using an iterator after `erase()` or `insert()`** — both operations invalidate all iterators at and after the modified position. Always use the iterator returned by `erase()` to continue the loop.
- ▶ **Comparing `v.size()` (unsigned) to a signed int with `<`** — cast to `int` or use `size_t` to avoid signed/unsigned comparison warnings.
- ▶ **Thinking `clear()` frees memory** — `clear()` sets size to 0 but the capacity remains. Use `vector<int>().swap(v)` to release memory completely.
- ▶ **Erasing inside a range-for loop** — this is undefined behavior. Use an index loop or an iterator loop with the return value of `erase()`.

6. Reflection Questions

Answer each question in 2–3 sentences:

- ▶ 1. What is the difference between `pop_back()` and `erase(v.end() - 1)`? When would you choose one over the other?
- ▶ 2. Why must you capture the return value of `erase()` when removing elements inside an iterator loop?
- ▶ 3. Both `clear()` and `resize(0)` produce a vector with size 0. What is the difference, if any?
- ▶ 4. Explain how `swap()` can exchange two vectors of different sizes in $O(1)$ time without moving any elements.
- ▶ 5. When would you use `emplace_back()` instead of `push_back()`? For `vector<int>` is there a practical difference?

Key concept

A vector is not just a dynamic array — it is a complete sequence container. Mastering these methods gives you the foundation to build the more complex data structures and algorithms studied in the rest of Chapter 17.